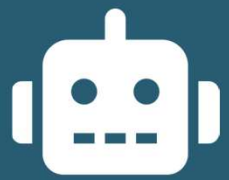
PROOF OF CONCEPT · SYSTÈME RAG

Assistant intelligent de recommandation d'événements culturels

Puls-Events × Open Agenda — Périmètre : Brest

Réponse aux exigences du POC — Jérémy, Puls-Events



Daniel — Ingénieur data freelance

Soutenance de projet — OpenClassrooms

CONTEXTE

Le contexte : la mission Puls-Events

Puls-Events

Plateforme web qui aide les utilisateurs à découvrir des événements culturels en temps réel, à partir de sources publiques comme Open Agenda.

L'objectif du POC

Tester un chatbot capable de fournir des recommandations personnalisées, appuyées sur un système RAG plutôt que sur les seules connaissances générales d'un LLM.

Périmètre géographique

Ville de Brest, retenue pour ce POC.

Fenêtre temporelle

Uniquement des événements de moins d'un an.

Source des données

API publique Open Agenda.

Attendu

Code versionné, tests, rapport, présentation, démo live.

Jérémy : Responsable technique

« Nous avons besoin d'une version fonctionnelle du POC RAG, sur les données publiques Open Agenda. Limite-toi au périmètre de ton choix, mais veille à n'avoir que des événements de moins d'un an. »

Livrables demandés :

- Environnement reproductible + README
- Système RAG (LangChain + Mistral + FAISS)
- Rapport technique (5-10 pages)
- Tests unitaires en pipeline
- Présentation + démo live

CONCEPT CLÉ

Qu'est-ce que le RAG ?

Un LLM seul

- Connaissances figées au moment de son entraînement
- Ne connaît aucun événement réel de Brest
- Risque d'halluciner : inventer une réponse plausible mais fausse

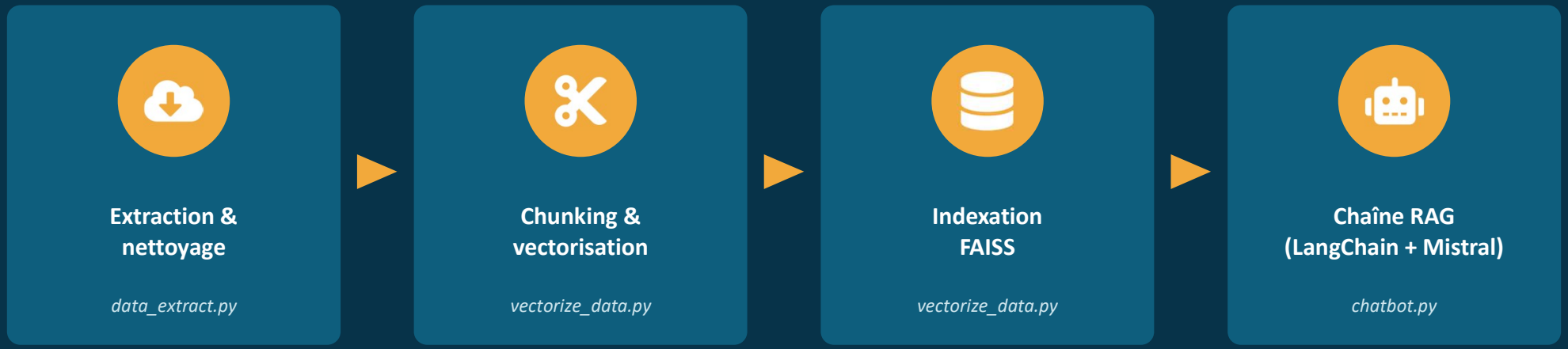
LLM + RAG (notre approche)

- On cherche d'abord dans une base de données réelle et à jour
- La réponse est construite à partir des événements trouvés
- Réponses sourcées, vérifiables, ancrées dans les vraies données

RAG = Retrieval-Augmented Generation. On augmente la génération du modèle de langage avec une étape de récupération d'information, effectuée juste avant de répondre.

ARCHITECTURE

L'architecture globale du pipeline



Chaque étape correspond à un script Python autonome. Le pipeline peut être rejoué à la demande pour reconstruire entièrement la base vectorielle.

ÉTAPES 1 & 2

Extraction et nettoyage des données

Connexion à l'API Open Agenda

Récupération des événements publics via pagination (100 résultats par page) jusqu'à épuisement des résultats.

Filtre dynamique

Ville = Brest, date ≥ aujourd'hui − 365 jours — recalculée à chaque exécution pour rester toujours juste.

Nettoyage systématique

Suppression des événements sans titre/description, et des doublons sur l'identifiant unique.

686

événements exploitables
après nettoyage

Ville de Brest — fenêtre d'un an

100% des événements retenus respectent les 2 critères métier — vérifié automatiquement par un test unitaire.

ÉTAPE 3

Chunking et vectorisation

Découpage (chunking)

Seule la description est découpée (500 caractères, chevauchement de 50). Le titre, le lieu et la date sont répétés dans chaque chunk pour ne jamais perdre le contexte de l'événement.

Vectorisation (embeddings)

Chaque chunk est transformé en vecteur numérique par le modèle mistral-embed, une représentation mathématique de son sens.

Indexation FAISS

Les vecteurs et leurs métadonnées (titre, lieu, date) sont stockés dans un index FAISS, prêt pour la recherche par similarité.

Sécurité

Clé API Mistral chargée depuis un fichier .env — jamais en dur dans le code.

Un embedding, visuellement

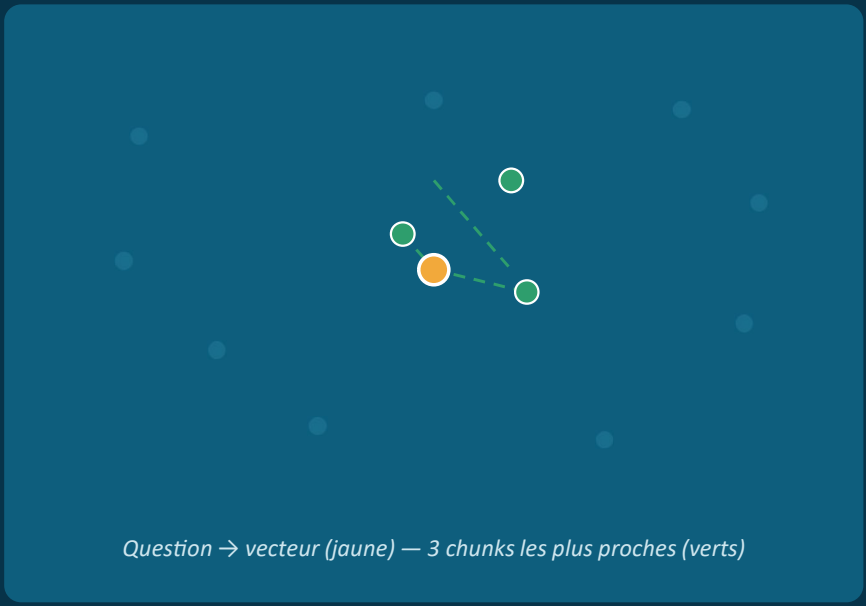


Chaque point = un chunk d'événement, placé selon son sens dans un espace à plus de 1000 dimensions.

FOCUS TECHNIQUE

FAISS : comment optimise-t-il la recherche ?

- Chaque chunk devient un point dans un espace à haute dimension (le vecteur)
- FAISS calcule la distance entre le vecteur de la question et tous les vecteurs indexés
- C'est une comparaison mathématique de proximité de sens, pas une recherche mot-clé
- L'algorithme retourne directement les k vecteurs les plus proches (k-plus-proches-voisins)
- Sur ce volume (≈1 245 vecteurs), la recherche exacte est quasi-instantanée



Question → vecteur (jaune) — 3 chunks les plus proches (verts)

Pourquoi FAISS ? Open source · Intégration native avec LangChain · Simplicité adaptée à un POC

FOCUS TECHNIQUE

FAISS : quelles limites à grande échelle ?

Aujourd'hui (POC — 1 245 vecteurs)

- Index IndexFlatL2 : recherche exacte, distance calculée point par point
- À cette échelle, le coût reste négligeable (quelques millisecondes)
- Le choix le plus simple et le plus fiable pour un POC

À grande échelle (millions d'événements)

- La recherche exacte devient linéaire : le temps de calcul augmente avec le volume
- Solution : index approximatifs (IVF, HNSW) — un peu moins précis, mais bien plus rapides
- Ou bases vectorielles distribuées (Milvus, Pinecone) pour scaler horizontalement

FOCUS TECHNIQUE

Pourquoi LangChain ?

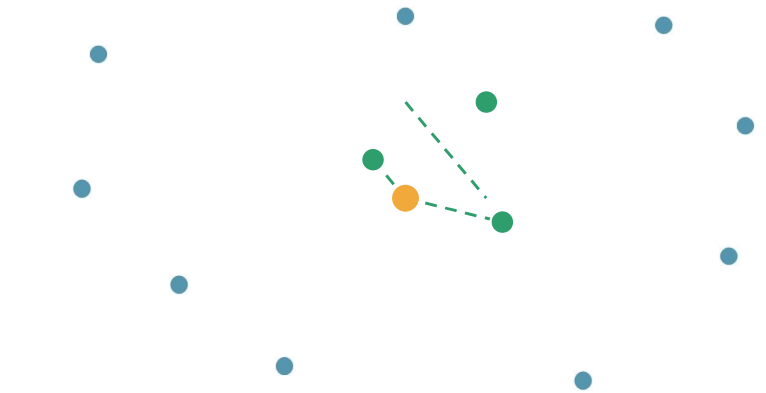


Une « chaîne » LangChain (LCEL) : la sortie d'une étape devient automatiquement l'entrée de la suivante.

- Intégration native avec FAISS et l'API Mistral — pas de code de liaison à écrire soi-même
- Syntaxe déclarative et lisible : chaque étape est un maillon relié par le symbole « | »
- Modularité : chaque brique (recherche, prompt, modèle) peut être remplacée indépendamment
- Écosystème riche et documenté, standard de facto pour orchestrer des pipelines RAG

QUESTION CLÉ

Comment le chatbot juge-t-il la pertinence ?



La question (jaune) est comparée à tous les chunks (bleus) — les plus proches (verts) sont sélectionnés.

1. La question est transformée en vecteur, exactement comme les chunks
2. FAISS calcule la distance entre ce vecteur et tous les vecteurs indexés
3. Les k chunks les plus proches (donc les plus proches en sens) sont sélectionnés
4. Les chunks d'un même événement sont dédupliqués (identifiant unique)
5. Le prompt interdit au modèle d'inventer si rien n'est vraiment pertinent

ÉTAPE 4

Le chatbot RAG en action



Pas d'historique de conversation

Choix volontaire du POC : chaque question est traitée indépendamment (cahier des charges).

Prompt de cadrage

Redirige poliment l'utilisateur si la question sort du périmètre (autre ville, hors sujet).

Température basse (0.2)

Le modèle reste fidèle au contexte fourni plutôt que de « créer » une réponse.

Aucune reconstruction

Le chatbot recharge l'index déjà construit — il ne revectorise jamais à chaque lancement.

ANNEXE · BACKUP POUR LA DISCUSSION

Le parcours complet d'une question

Exemple : « Y a-t-il un concert de rock ce soir ? »

LangChain — orchestrateur · Mistral — traduction & rédaction · FAISS — recherche vectorielle

1 · LangChain reçoit la question

rag_chain.invoke(question) déclenche toute la chaîne.

2 · Mistral-embed traduit en vecteur

La question devient un vecteur numérique, pour capter le concept « musique rock ».

[0.4, -0.8, 0.1 ...]

3 · FAISS calcule les distances

Comparaison avec tous les vecteurs indexés → les 3 chunks les plus proches (dédupliqués par événement).

4 · FAISS renvoie texte + métadonnées

Chaque vecteur gagnant reste relié à son texte original ET à ses métadonnées (titre, lieu, date) depuis l'indexation.

5 · LangChain assemble le prompt

Consignes strictes + les 3 textes trouvés + la question, dans un seul gabarit.

6 · Mistral-small rédige la réponse

Lit les 3 événements fournis et construit une réponse en français, sans rien inventer.

7 · L'utilisateur reçoit la réponse

« Ce soir, concert de rock au Quartz... »

RÉSULTATS

Les résultats obtenus pour le POC

686

événements exploitables

1 245

chunks vectorisés et indexés

4 / 4

tests unitaires passants

Exemple concret : requête « concert de musique gratuit »

1. **La Nuit des conservatoires à Brest**
   Conservatoire de Brest métropole — vendredi 30 janvier, 17h30
2. **Cinérencontre — Soundtrack to a Coup d'État**
   Cinéma Les Studios — dimanche 28 septembre, 16h00
3. **Concert pop/rock avec Stellen**
   Maison ancienne années 30, Brest — mardi 14 avril, 18h00

→ *3 événements distincts et pertinents renvoyés — aucun doublon.*

FIABILITÉ

Qualité des données et limites actuelles

Ce qui garantit la qualité

- Nettoyage automatique (valeurs manquantes, doublons)
- Filtre de date recalculé dynamiquement à chaque exécution
- Pipeline de tests pytest exécuté à chaque changement
- Clé API sécurisée via .env, jamais en dur dans le code

Limites actuelles du POC

- Pas d'historique de conversation (choix assumé du POC)
- Pas d'évaluation quantitative formalisée à ce stade
- Index non incrémental : reconstruction complète à chaque mise à jour
- Pas de seuil de similarité ni de re-ranking des résultats

VERS LA PRODUCTION

Recommandations pour la suite

Mode 2 : filtrage intelligent

Détecter des expressions comme « ce soir » ou « demain » pour ne chercher que dans les événements à venir, pas tout l'historique.

Automatisation du pipeline

Orchestrer data_extract.py avec un outil comme Kestra pour qu'il tourne seul chaque matin, sans lancement manuel.

Migration vers le Cloud

Héberger la base FAISS et le JSON sur AWS S3 pour passer à l'échelle et rendre l'appli accessible de partout.

Interface graphique (Streamlit)

Remplacer le terminal par une vraie page web type chat, plus ergonomique et plus vendeuse pour le client final.

Place à la démonstration

- Une question précise → l'événement exact est retrouvé, avec lieu et date
- Une question vague (« que faire ce week-end ? ») → plusieurs suggestions pertinentes
- Une question hors périmètre → le bot recadre poliment plutôt que d'inventer

---

Merci de votre attention